

# Refactoring e implementazione di servizi di Machine Learning per l'analisi dati in un contesto distribuito basato su Kubernetes

Gabriele Stentella  
984491

## Relatore

Prof. Claudio A. Ardagna

## Correlatori

Prof. Marco Anisetti

Dott. Antongiacomo Polimeno

---

Nel contesto della moderna elaborazione dei dati, l'intelligenza artificiale (IA) e il *cloud computing* rappresentano due pilastri fondamentali per la trasformazione digitale delle aziende, delle organizzazioni pubbliche e degli istituti di ricerca. L'IA, attraverso algoritmi avanzati di *machine learning* (ML) e *deep learning* (DL), consente di analizzare grandi quantità di dati in modo efficiente, identificare pattern e trend nascosti, e assistere nelle decisioni basandosi su modelli statistici. Questo approccio permette di ottenere informazioni di valore a partire da dati non strutturati con elevata dimensionalità, eterogeneità e presenza massiccia di correlazioni spurie, detti *big data*.

Con il paradigma del cloud computing è possibile accedere ad un'infrastruttura flessibile e scalabile per archiviare, elaborare e distribuire i dati necessari per gli algoritmi di IA. Grazie al cloud, si può accedere a risorse informatiche *on-demand*, riducendo i costi di gestione e manutenzione dei server fisici. Inoltre, consente di implementare, in modo rapido ed efficiente, soluzioni che richiedono grande potenza di calcolo, dunque può essere sfruttato per addestrare modelli complessi e gestire carichi di lavoro intensivi.

Sebbene attraverso il cloud si possa accedere a risorse molto ampie, per svolgere efficientemente analisi su insiemi di dati ad elevata dimensionalità è necessario parallelizzare il carico computazionale. Nel calcolo distribuito, l'obiettivo è quello di dividere sia i dataset sia le operazioni da svolgere sui dati su diversi nodi di un *cluster*. Un cluster può essere implementato con architetture molto varie, sia utilizzando macchine fisicamente distinte, sia nodi implementati in un ambiente virtualizzato.

Con l'avvento dei big data sono nati dei framework pensati per gestire sia la distribuzione di dati su più dispositivi, sia l'orchestrazione delle operazioni che vengono svolte su uno stesso insieme di dati, ma su nodi diversi. Un primo approccio al calcolo distribuito è stato quello proposto con la tecnologia *MapReduce*, ma successivamente è stato sviluppato *Apache Spark* che consente di leggere i dati dalla memoria, eseguire operazioni e scrivere i risultati in un singolo step. Spark ha inoltre introdotto la possibilità di riutilizzo dei dati mantenuti in cache, caratteristica che consente una grande ottimizzazione di algoritmi che chiamano ripetutamente una funzione su uno stesso dataset, come quelli di machine learning. Fra le API offerte da Spark si trova infatti una libreria specificatamente dedicata al machine learning (MLlib), ampiamente utilizzata in questo lavoro di tesi. Un ulteriore vantaggio del framework Spark è quello di non avere un proprio sistema di archiviazione, ma di permettere l'utilizzo sia di sistemi esterni come HDFS o *Amazon S3*, sia di soluzioni personalizzate.

L'obiettivo di questa tesi è quello di recuperare algoritmi di machine learning per l'analisi dati sviluppati come *Spark tasks* in linguaggio Java, e studiare l'implementazione di nuovi algoritmi seguendo un approccio più adatto a modelli di DL, quindi utilizzando il linguaggio *Python* e la libreria *Tensorflow*. In particolare, le attività di recupero e implementazione mirano ad ottenere singoli servizi che implementano modelli per l'analisi dati. Per rendere autonoma l'esecuzione dei servizi, si utilizza la tecnologia dei *container Docker*, ossia unità software standardizzate per la virtualizzazione. I servizi che implementano gli algoritmi sono eseguiti in un contesto distribuito gestito dall'orchestratore *Kubernetes*.

La motivazione del lavoro è da vedersi nell'evidente vantaggio che deriva dalla possibilità di disaccoppiare il codice dall'infrastruttura su cui viene eseguito. Attraverso il refactoring di algoritmi pre-esistenti e l'implementazione di nuovi, si è studiata la possibilità di eseguire i servizi software su un'infrastruttura distribuita diversa da quella per cui erano stati originariamente pensati. Un aspetto fondamentale del metodo

presentato nella tesi consiste nelle minime modifiche operate sul codice, talvolta del tutto assenti. Un ulteriore contributo di questo lavoro è dato dalla possibilità di integrare i servizi implementati con Tensorflow all'interno di un cluster pensato per l'esecuzione di applicazioni Spark.

Il lavoro di recupero degli algoritmi è stato portato avanti parallelamente allo studio dell'implementazione di nuovi servizi, e l'attività svolta durante il tirocinio può essere riassunta dalle fasi seguenti.

- **Analisi degli algoritmi esistenti.** È stata effettuata un'analisi dei diversi algoritmi di ML già implementati al fine di individuare quelli più significativi per l'analisi dei dati. Successivamente, ogni algoritmo selezionato è stato analizzato in dettaglio per individuarne le dipendenze e le versioni delle diverse tecnologie utilizzate.
- **Aggiornamento e riorganizzazione degli algoritmi.** Gli algoritmi selezionati sono stati aggiornati per utilizzare la versione più moderna di Spark. Si è anche reso necessario l'aggiornamento di una libreria esterna non più mantenuta, ma utilizzata dagli algoritmi. La struttura dei progetti dei singoli task è stata ripensata in modo tale da poter costruire i diversi file archivio (jar), rispettando le dipendenze presenti fra i task.
- **Implementazione di un algoritmo DL in Tensorflow.** È stato sviluppato un modello predittivo di DL in Python utilizzando la libreria Tensorflow.
- **Sviluppo dei servizi.** I modelli ML e DL sono stati inseriti in *container Docker* per consentirne la successiva esecuzione in un contesto distribuito.
- **Esecuzione distribuita degli algoritmi.** Si sono studiate le modalità di distribuzione del carico computazionale sia dei servizi sviluppati con Java e Spark sia dell'algoritmo implementato in Tensorflow. Tutti i modelli sono stati eseguiti su un cluster Kubernetes virtualizzato su una singola macchina. In seguito è stato implementato e utilizzato un secondo cluster in cloud utilizzando le tecnologie *Amazon EMR on EKS* e *Amazon S3*.
- **Esecuzione sull'infrastruttura MUSA.** Il *deployment* finale è stato fatto in un ambiente distribuito, gestito con Kubernetes, nell'infrastruttura sviluppata nell'ambito del progetto MUSA, finanziato con i fondi del PNRR da parte del Ministero dell'Università e della Ricerca.
- **Test di performance dell'esecuzione distribuita.** Sono stati analizzati i dati di log dei task Spark e quelli dell'allenamento del modello sviluppato con Tensorflow, al fine di dimostrare l'effettiva distribuzione del carico computazionale e dunque i vantaggi dell'esecuzione in contesto distribuito.

La tesi lascia spazio per sviluppi futuri. Primo fra tutti, l'utilizzo di un sistema di *scheduling* dei flussi di lavoro come *Apache Oozie* o *Airflow* per creare delle *pipeline* di task in modo tale da costruire un'infrastruttura idonea ad analisi dati più complessa. In secondo luogo, è da notare che l'approccio proposto per l'implementazione di nuovi algoritmi utilizzando Python e Tensorflow è scalabile, quindi resta aperta la possibilità di sviluppare nuovi modelli per l'analisi dati, o rendere più efficienti algoritmi già implementati e noti in letteratura.